

PROGRAMMING FOCUS TRACK

Name: Vladut-Andrei Lambru 487791

Class: 2B

Teacher: Armando Gerard

Submission date: 29.01.2026

Table of Contents

PROGRAMMING FOCUS TRACK	1
1. Introduction	2
2. Personal Learning Goals	3
3. Research	3

4. Requirement List	4
5. Technical Design	5
5.1 XP Gain and Level Up System	7
5.2 Upgrade Menu.....	8
5.3 Upgrade Application	9
5.4 Upgrades: Dash and Double Dash	10
5.5 Enemy Respawn (NavMesh random spawn).....	10
5.6 Upgrades: Double Jump	11
5.7 Upgrades: Movement Speed on Kill.....	11
5.8 Upgrades: Overheat Tuning	12
5.9 Upgrades: Critical Hits	12
6. Early Development mistakes:	13
7. User Guide	13
8. Reflection	15
9. Bibliography	17

1. Introduction

I chose this case for two specific reasons. Firstly, I wanted to build something that feels like a real game feature, not just scripts that add numbers. I've always been interested in how progression systems work in games, and I wanted to understand how to make one that actually integrates properly with existing code rather than just sitting on top of it as a separate thing.

Secondly, I often find myself struggling with creating something scalable. When I look back at code I wrote a few months ago, I can see how messy it gets when I don't plan properly. I realized I needed to learn how to build systems that can be expanded without rewriting everything from scratch.

In general, I think I'm pretty good at rapid prototyping and creating something quick to test out a mechanic, but I often ignore the growing complexity and unreadability of my code, which would be a problem if I started working with other programmers. I realized

that I still needed to learn proper design patterns and workflows to make sure that I could not only create something that works in the short term, but also something that can be built upon and used as a core system without running into a lot of issues.

2. Personal Learning Goals

Integration: Create a system that works with the existing code and follows the existing workflow, rather than existing as a separate entity.

Scalability: Create a system that is easily scalable and allows for expansion, without drastically increasing complexity.

Efficiency and Readability: Learn how to make my code work efficiently despite the system's complexity, as well as being more readable to allow co-working and easier bug-fixing and troubleshooting.

3. Research

The main inspiration for the feature is Assassin's Creed Origins, where the player collects XP points through various actions, receiving a "skill point" at a certain threshold, and then can spend those skill points to unlock new abilities that are divided into different "branches", suiting different playstyles.

There are three basic components of this level-up system: The method to gain XP, the conversion method to use the XP to receive rewards (upgrades), and the upgrades themselves.

For the method to gain XP, this feature follows the base system found in a lot of RPGs like Final Fantasy, Monster Hunter, or even tabletop games like Dungeons and Dragons. These games award XP points when killing enemies, with each enemy awarding a set amount. I looked at a few articles about leveling formulas to understand how thresholds work and how to make progression feel consistent.

For the conversion method, I looked at how games like Shadow of the Tomb Raider and Yakuza 0 handle skill trees. These games award points at every level up and then the player can spend those points on upgrades. I decided to simplify this for my system; instead of a full skill tree with branches, I went with a card-based choice system where

you pick between two options (movement or shooting) each time you level up. This keeps it simpler and faster.

The upgrades themselves are inspired by many different games, for example:

Dash and movement speed from Ghostrunner, where dash movement completely changes how you approach combat. Double Jump from Titanfall 2, which opens up vertical movement. Overheat mechanics from sci-fi shooters where weapons use heat instead of traditional ammo. Critical hits from looter shooters like Borderlands, where you get clear feedback when you land a high-damage shot.



4. Requirement List

XP Gain:

The player can collect XP by killing enemies, which is shown as a value on the UI. Once a certain threshold of XP is collected (which increases with each level), the player gains a level and the upgrade menu opens automatically.

Menu and Upgrade Management:

In the upgrade menu, the player sees two cards: one for a movement upgrade and one for a shooting upgrade. The player can click on one of the cards to select it, which

applies the upgrade immediately. The menu then closes and gameplay resumes. The selected upgrade is saved in the player's state.

Upgrades:

Movement Branch: Dash (gives the player a burst of speed with a cooldown), Double Jump (allows one jump in mid-air), Adrenaline Rush (movement speed boost after killing an enemy).

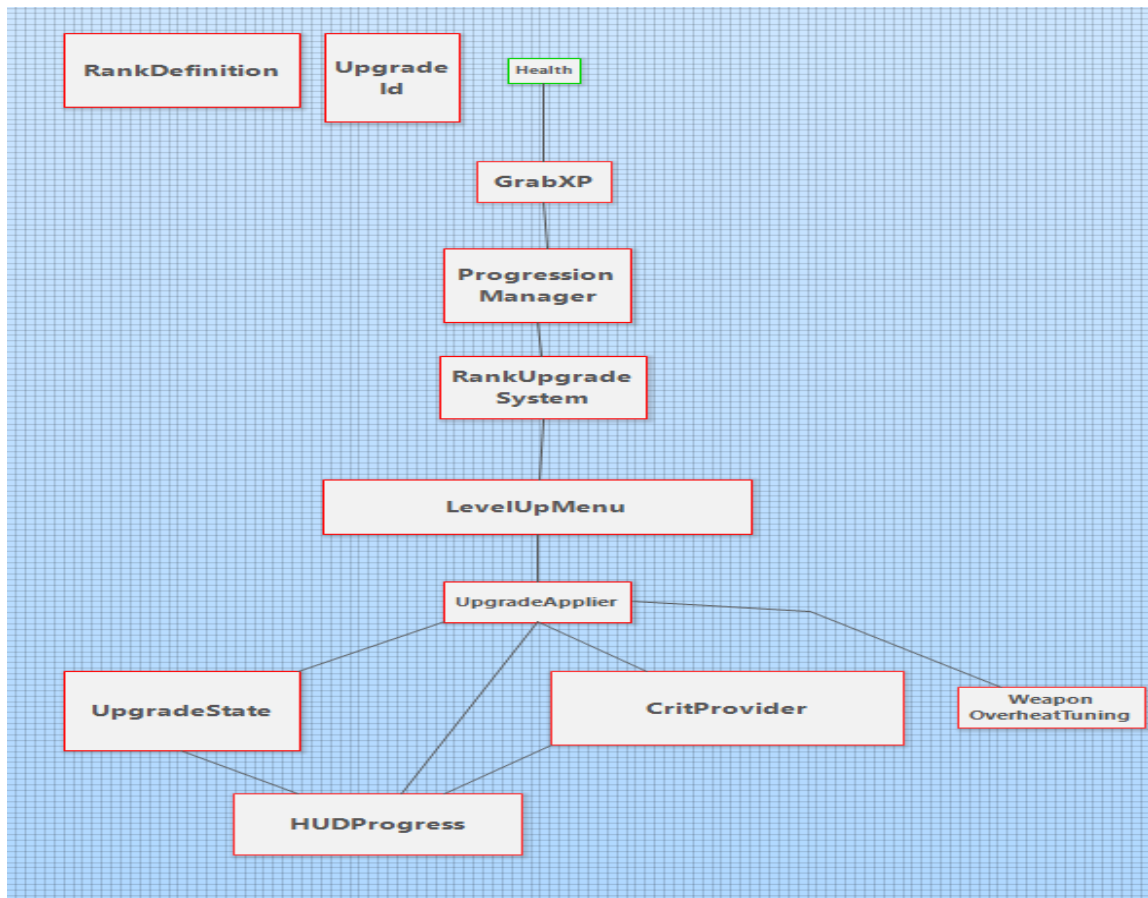
Shooting Branch: Overheat Tier 1 (improves weapon cooling rate), Overheat Tier 2 (further improves cooling and recovery), Critical Chance (adds a chance to deal critical damage with visual feedback).

5. Technical Design

General UML Flowchart:

Red Classes and Associations were created completely from scratch. Yellow Classes were already present but were modified to add new code. Green Classes and

Associations were already present and not modified, just used.

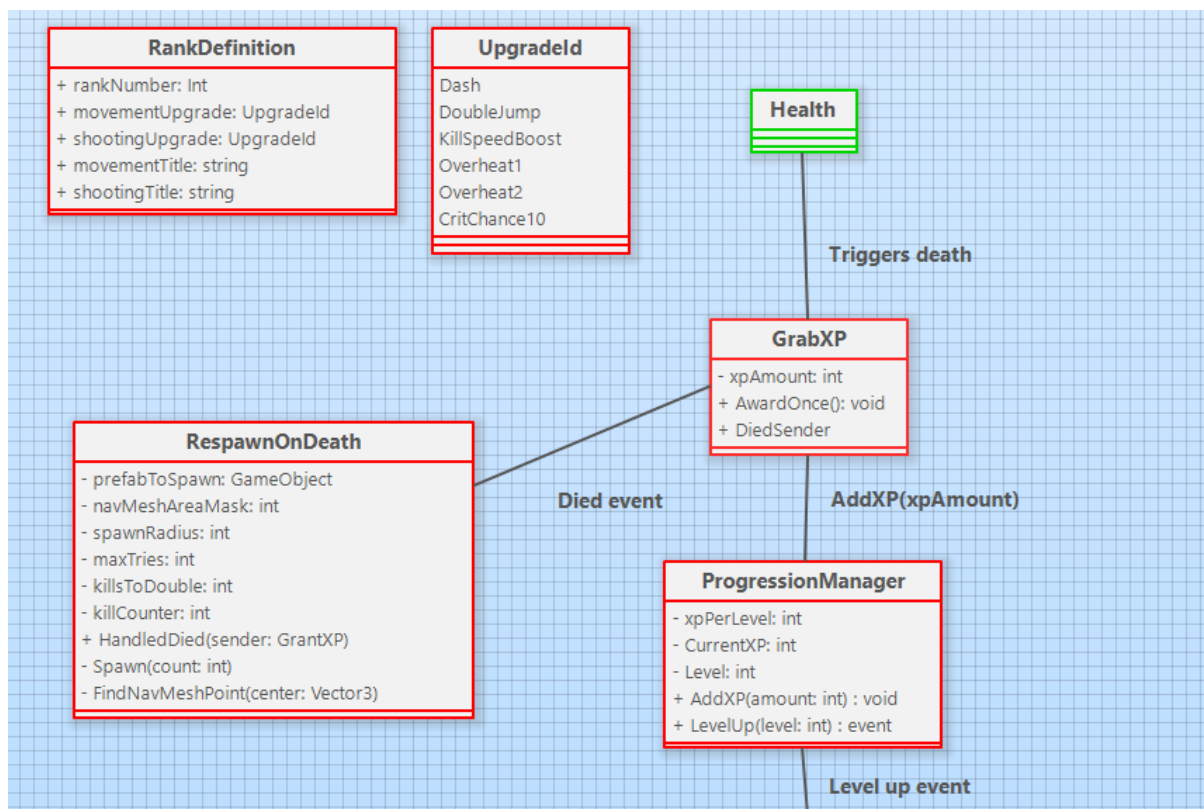


ProgressionManager: Contains the current XP value, the player's level, and the XP threshold for leveling up. When AddXP is called, it adds the XP amount and checks if the

threshold has been reached. If yes, it fires a LevelUp event and calculates the new threshold, carrying over any extra XP.

Iterations:

Made sure that XP carry-over worked correctly when leveling up. If you earn 50 XP but only need 30 to level, the extra 20 should roll into the next level. Added a simple linear formula for the threshold (xpPerLevel multiplied by current level). Used Unity's event system to keep ProgressionManager decoupled from the upgrade menu.



5.2 Upgrade Menu

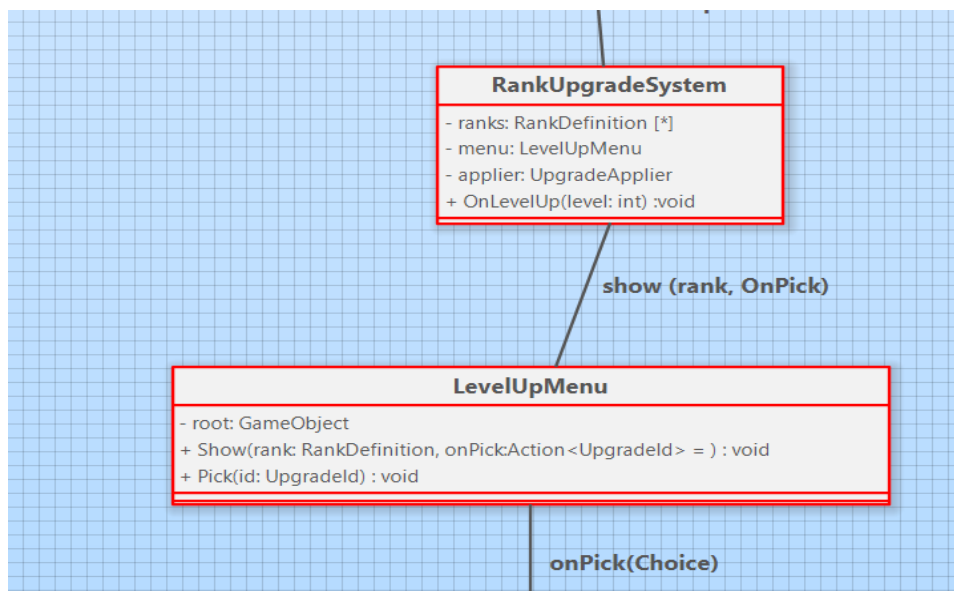
RankUpgradeSystem: This script listens for the LevelUp event from ProgressionManager. When it fires, it looks up the correct RankDefinition based on the player's level and tells the LevelUpMenu to open with that rank's data.

RankDefinition (ScriptableObject): This is where all the rank data is stored. Each rank has a movement upgrade option (title, description, upgrade ID) and a shooting upgrade option (title, description, upgrade ID). This makes it really easy to add new ranks; you just create a new ScriptableObject asset, fill in the fields, and add it to the ranks array.

LevelUpMenu: This is the UI controller. It receives the RankDefinition, updates all the TextMeshPro text fields with the titles and descriptions, pauses the game (Time.timeScale = 0), and unlocks the cursor. When the player clicks one of the buttons, it calls a callback function with the selected upgrade ID, closes the menu, and resumes the game.

Iterations:

Left the first rank (rank 0) unused to make the IDs easier to read and remember. Used the existing pause functionality from the template instead of creating a completely new system. Initially had issues with the menu not responding to mouse input because the template's pause menu was still active. Fixed this by making sure only one UI system is active at a time and explicitly unlocking the cursor when the menu opens.



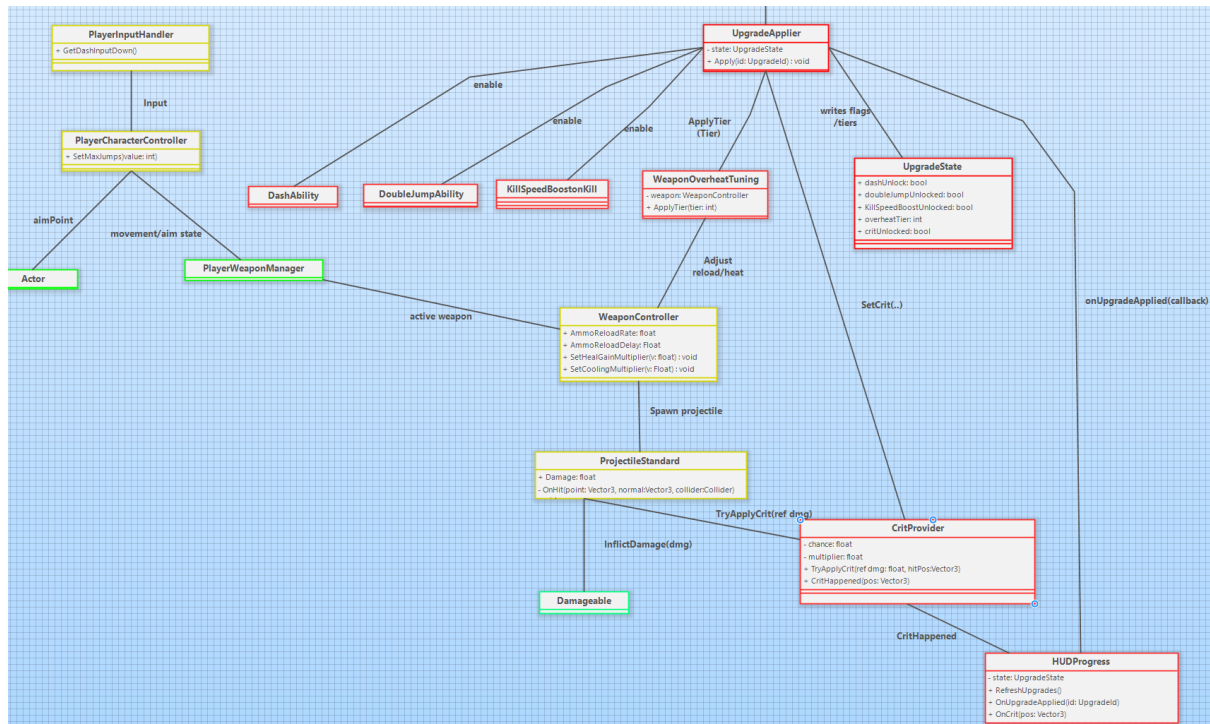
5.3 Upgrade Application

UpgradeApplier: This is the bridge between the menu and the actual gameplay. It receives the upgrade ID from the menu and then enables the appropriate scripts or sets the appropriate values. For example, if the ID is "dash", it enables the DashAbility script on the player. If the ID is "overheat_tier1", it applies tuning values to the WeaponOverheatTuning component. It also updates the UpgradeState so the HUD can display which upgrades are unlocked.

UpgradeState: A simple data class that tracks which upgrades the player has unlocked. This is used by the HUD to display the list of active upgrades.

Iterations:

Started with all the upgrade logic in one massive switch statement, then broke it down into smaller helper functions to make it more readable. Added checks to make sure all the required component references exist before trying to enable them, to avoid null reference errors.



5.4 Upgrades: Dash and Double Dash

DashAbility: This script is attached to the player and handles the dash mechanic. When the player presses the dash input (Q key), it gives them a burst of speed in the direction they're moving. The dash uses a coroutine to apply the movement force over a short time, then goes on cooldown.

Iterations:

Used the existing input system rather than `Input.GetKeyDown` to keep everything consistent. Made the dash a coroutine to simplify the timing logic. Added a check to stop the previous dash before starting a new one, to avoid stacking movement. Tuned the dash force and cooldown values multiple times to make it feel punchy and satisfying.

5.5 Enemy Respawn (NavMesh random spawn)

Goal:

The game needed an endless combat loop: **when a HoverBot dies, a new HoverBot spawns**, and when a Turret dies, a new Turret spawns. Spawns must be **random**, but

still valid gameplay-wise, so enemies should appear only on **walkable NavMesh surfaces** (no floating, inside walls, or outside map).

Final approach (what I implemented):

I created a reusable component (**RespawnOnDeathNavMesh**) that is added directly to each enemy prefab/root.

- The enemy already has **GrantXPOnDeath**, which fires a **Died** event when the enemy is confirmed dead.
- **RespawnOnDeathNavMesh** subscribes to that event and triggers a respawn.
- On death, it spawns **1 new enemy** by default.
- Every **X kills** (example: 5), it increases the amount spawned (e.g., spawn **2** instead of 1) to scale difficulty over time.
- New enemies spawn at a **random NavMesh point** within a radius around a chosen center (or around the old enemy position / a global origin), using NavMesh sampling so the spawn point is guaranteed to be valid.

5.6 Upgrades: Double Jump

DoubleJumpAbility: This modifies the existing jump code to allow one mid-air jump. The ability tracks whether the player has used their mid-air jump and resets it when they touch the ground.

Iterations:

Put the jump code into its own method to avoid repetition. Integrated the mid-air jump logic into the existing air movement code from the template. Made sure the second jump only fires when the player is actually in the air, not when they're still on the ground. Fixed an issue where both jumps would sometimes fire at once.

5.7 Upgrades: Movement Speed on Kill

KillSpeedBoostOnKill: This script listens for enemy death events. When an enemy dies, it starts a coroutine that multiplies the player's movement speed for a few seconds.

Iterations:

Integrated the speed boost into the existing event system. Made it a coroutine to handle the duration cleanly. Changed how the speed multiplier is applied to mimic the existing sprint code from the template.

5.8 Upgrades: Overheat Tuning

WeaponOverheatTuning: This script modifies the weapon's overheat behavior. Tier 1 improves the cooling rate, and Tier 2 improves both cooling and recovery time. This changes the rhythm of combat; you can shoot for longer before overheating and recover faster when you do overheat.

Iterations:

Tested different tuning values to find ones that felt impactful. The original values barely made a difference, so I increased them until the upgrade was clearly noticeable during gameplay.

5.9 Upgrades: Critical Hits

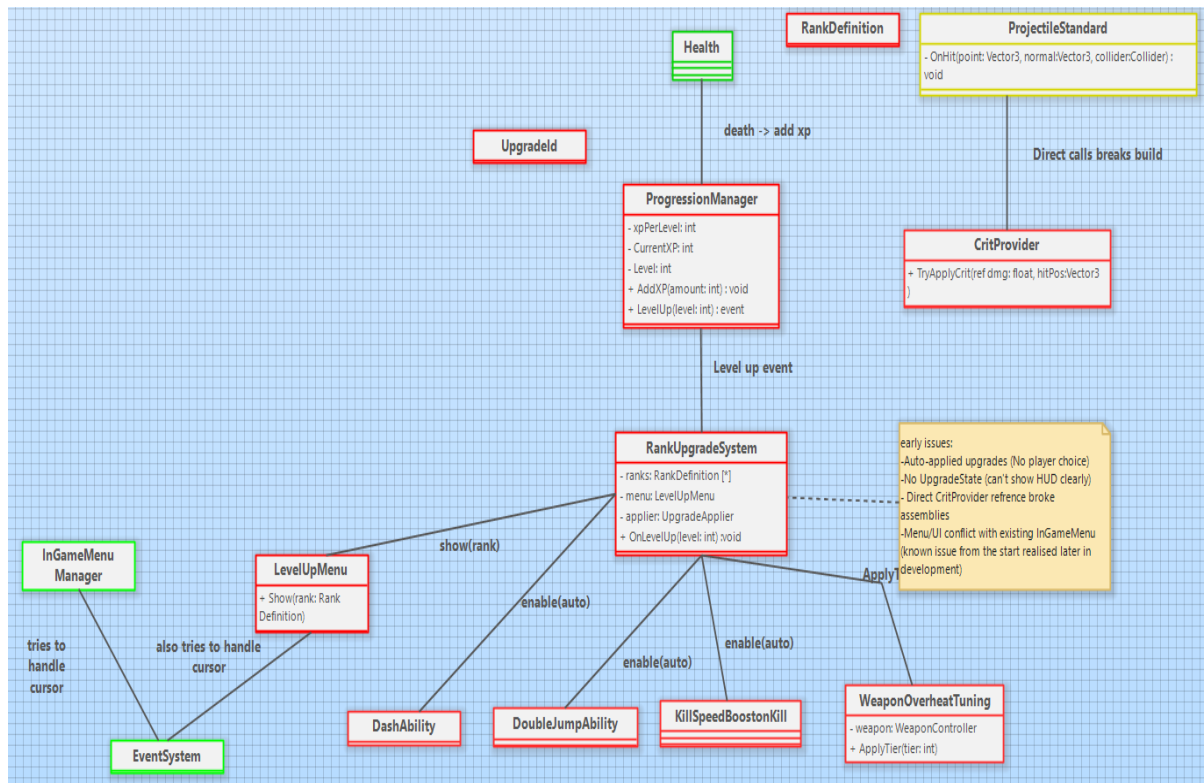
CritProvider: This script is attached to the player's weapon. When a shot is fired, it rolls a random chance to see if it's a critical hit. If it is, it multiplies the damage and fires an event that the HUD listens to.

ProgressionHUDSimple: Listens for the CritHappened event and displays a "CRIT!" toast message on screen for a couple of seconds.

Iterations:

Used `Random.value` instead of `Random.Range(0,1)` for cleaner code. Added the HUD toast feedback because without it, players didn't know when crits were happening. Tuned the crit chance and damage multiplier to make it feel rewarding without being overpowered.

6. Early Development mistakes:



7. User Guide

Upgrade Menu:

The Upgrade Menu opens automatically when you level up. The menu shows two cards: one for a movement upgrade and one for a shooting upgrade. Click on one of the cards to select it. The upgrade is applied immediately and the game resumes.

Receiving XP and Leveling Up:

XP is awarded when you kill enemies. The amount depends on the enemy type (for example, hover bots give 30 XP). When your XP reaches the threshold, you level up and the upgrade menu opens. Any extra XP carries over to the next level.

Upgrade: Dash:

Press the Q key to dash in the direction you're moving. The dash gives you a burst of speed and then goes on a short cooldown.

Upgrade: Double Jump:

Press the spacebar while in mid-air to perform a second jump. The ability recharges when you touch the ground.

Upgrade: Movement Speed on Kill:

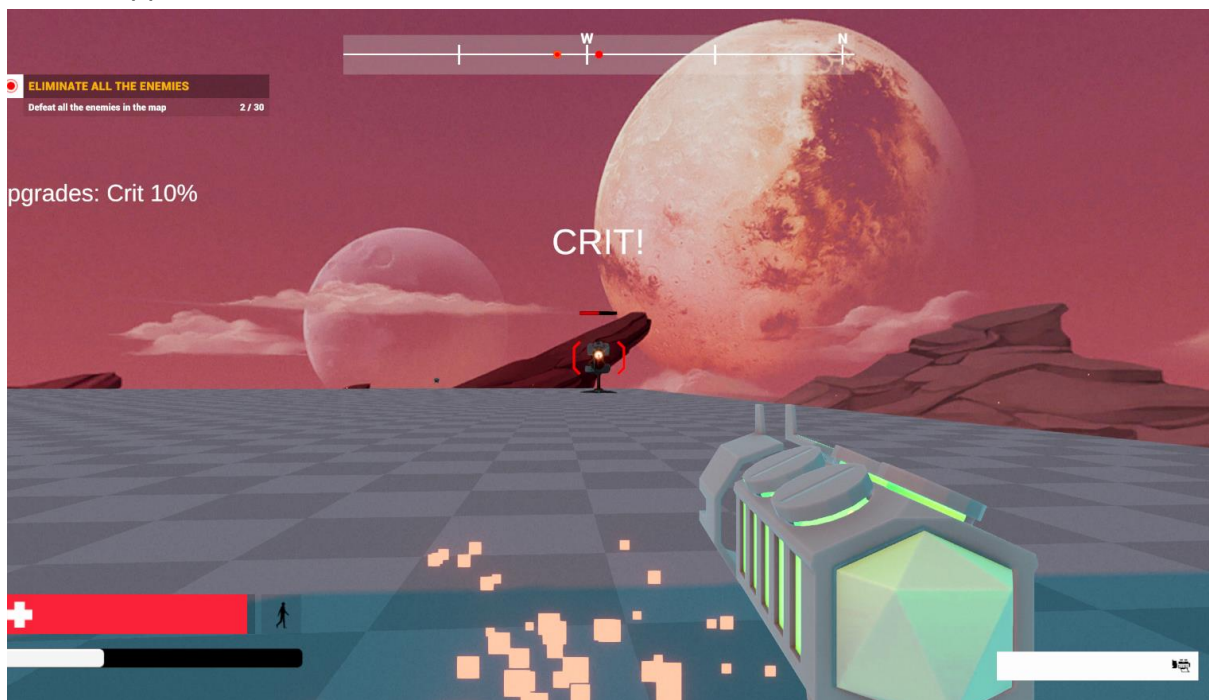
After killing an enemy, your movement speed is boosted for a few seconds.

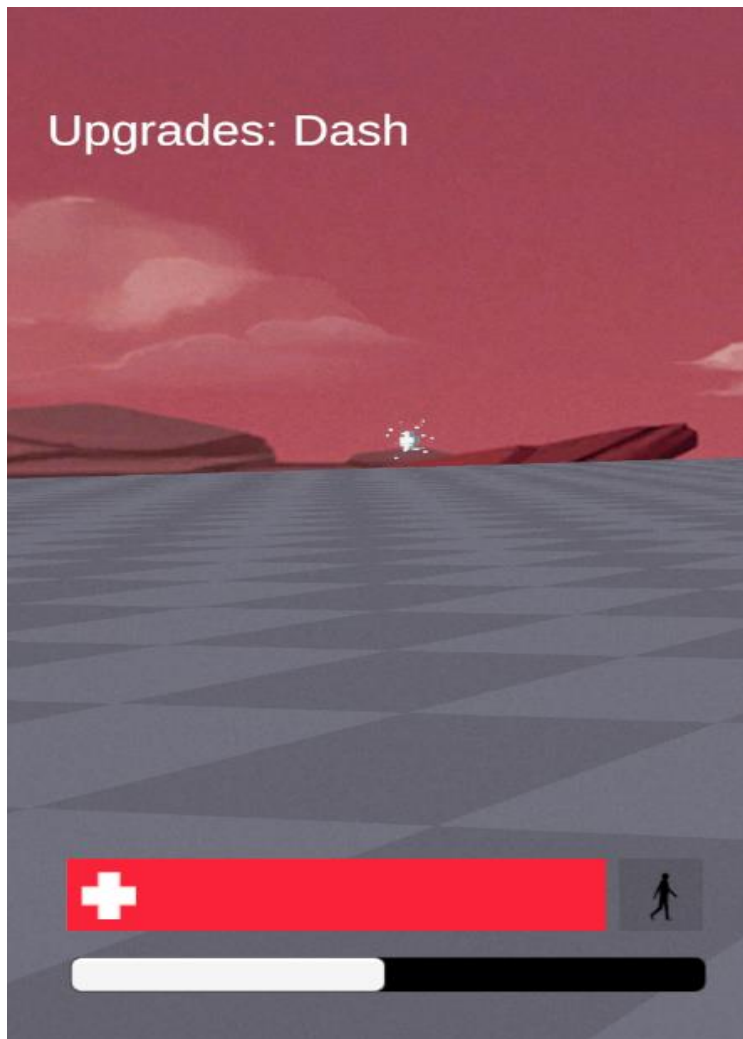
Upgrade: Overheat Tier 1 and Tier 2:

Your weapon cools down faster after firing. Tier 2 also makes you recover from overheat faster.

Upgrade: Critical Chance:

Your shots have a chance to deal critical damage. When a crit happens, you'll see "CRIT!" appear on the HUD.





8. Reflection

With the project finished, I can now see that I improved a lot in the areas I felt I had the most weaknesses. The differences between how I used to code and how I coded this

system are pretty clear: my code is now more efficient, I understand some patterns to use and some mistakes to avoid, and my overall workflow has improved with proper research and technical design phases.

My goal of creating a scalable system that can work as a basis to be expanded and iterated upon has been reached. I think that using this type of approach when designing systems will be very beneficial for me in the future.

Learning Goals:

Integration: I created a system that works using the existing codebase and follows the existing workflow. The menu uses the template's pause functionality, and all of the upgrades work through already existing systems. This means the system exists within the original code and not alongside it.

Scalability: The system I created is designed to work with as few or as many upgrades as needed. Adding new ranks is just creating a new ScriptableObject and adding it to the array. The system could be expanded to work in a larger game without major rewrites.

Efficiency and Readability: I learned how to make my code run more efficiently and be more readable. From small things like using proper naming conventions and adding tooltips, to larger changes like using events and separating concerns clearly. This will be crucial when working in larger projects and larger teams.

Things to Improve:

There is, however, still much I have to learn and there are more improvements I can make to my programming.

Design Patterns: In this project, I used some design patterns like the observer pattern for events and data-driven design with ScriptableObjects, but I would still want to learn and get used to many more of them. There are probably patterns that could make some parts of the code even cleaner.

UI System Integration: The hardest part of this project was dealing with UI input conflicts. The template already had UI systems, and making my menu work alongside them without breaking things took a lot of debugging. I'd like to learn more about Unity's UI architecture to handle this better in the future.

Debugging: I spent a lot of time debugging issues that could probably have been caught earlier with better debugging practices. Learning to use Unity's debugging tools more effectively is definitely something on my priority list.

As a final statement, I believe I learned a lot during this project. Having a proper workflow, using UML diagrams, and taking advantage of design patterns are all things I will use in my projects going forward.

9. Bibliography

Unity Documentation - UI EventSystem. Retrieved from Unity:

<https://docs.unity3d.com/>

Unity Documentation - ScriptableObjects. Retrieved from Unity:

<https://docs.unity3d.com/>

Game Programming Patterns - Observer Pattern. Retrieved from

<https://gameprogrammingpatterns.com/>

Assassin's Creed Origins (2017). Ubisoft Montreal.

Ghostrunner (2020). One More Level.

Titanfall 2 (2016). Respawn Entertainment.